



USER MANUAL

ClaudeManager v4.0.0

User Manual

V4.0.0

APRIL 2026

COMPREHENSIVE GUIDE

REACH

Introduction

ClaudeManager is a self-hosted dashboard for orchestrating, monitoring, and communicating with multiple Claude Code AI agent sessions in real time. It enables you to manage teams of AI agents working across different projects, all from a single interface accessible on desktop, mobile, and the native Android app.

This manual covers every feature of ClaudeManager v4.0.0, from basic agent monitoring to advanced multi-agent project orchestration with effort and model controls. Whether you are running a single Claude session or coordinating a team of specialised agents across multiple projects, this guide will show you how.

What's New in v4.0.0

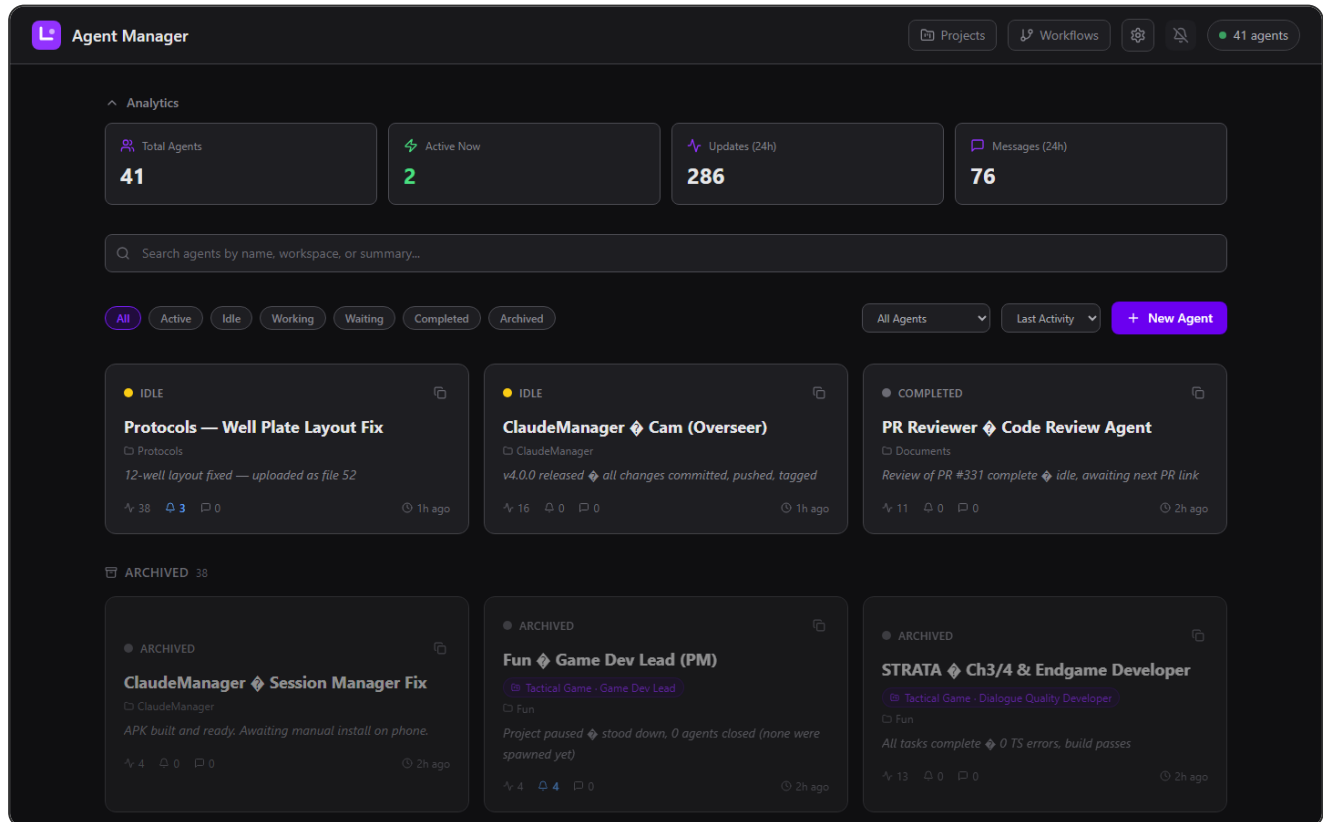
FEATURE	SUMMARY
Agent relay	Agents send structured messages to each other; relay history shown in timeline with direction arrows
Effort & model limits	Per-project caps on sub-agent model (haiku/sonnet/opus) and effort level, enforced server-side
PM role customisation	Assign a domain specialisation to the PM (e.g. "Machine Learning Engineer") while keeping its management identity
Smart project pause	Pausing a project delivers a notice to every active agent and cancels pending launch requests
Role/task agent launch	New-agent flow is now two steps: folder picker then role and task assignment
Per-agent cost tracking	Token spend tracked per agent with a dedicated cost breakdown panel
Watchdog v4	Process-level monitoring, overnight persistence, no PowerShell flashes; auto-starts on Windows boot
Android improvements	Cost tab, relay timeline, hold-to-copy full content, project management with effort/model controls
5-second message polling	Background watcher replaced 60-second intervals — messages delivered in under 5 seconds

Who Is This For?

- **Developers** using Claude Code for software engineering tasks who want centralised monitoring and remote control
- **Team leads** coordinating multiple AI agents across projects with cost and resource visibility
- **Researchers** running long-duration experiments with Claude agents that need periodic check-ins and instruction updates

The Dashboard

The dashboard is your central command centre. It shows all registered agents with their current status, latest activity, and quick actions.



Navigation

The top navigation bar provides access to the main sections:

SECTION	PURPOSE
Agents	List all agents with status, search, sort, and filter
Projects	Manage multi-agent project orchestrations with effort/model controls
Workflows	Define and run automated agent pipelines
Settings	API key, retention policies, webhooks, push notifications

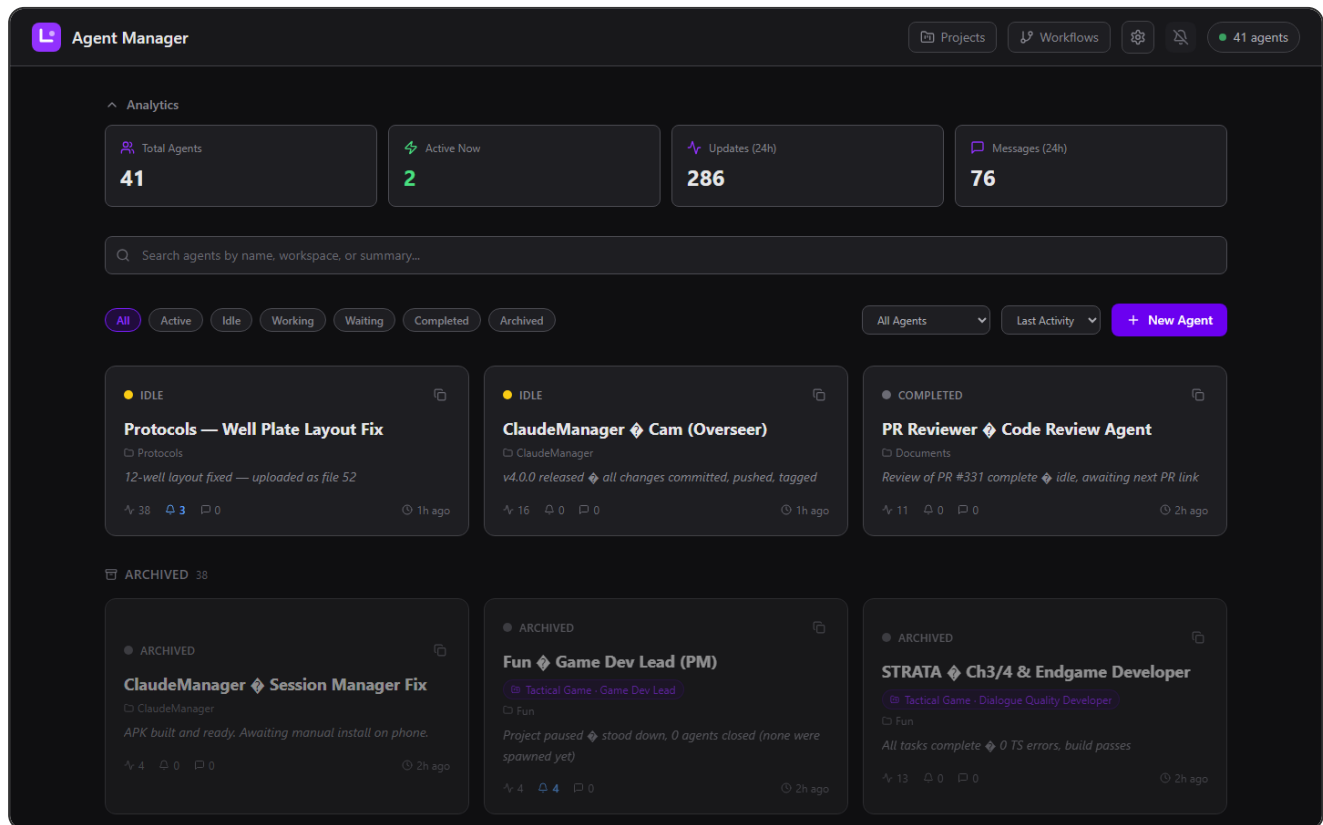
Analytics Panel

At the top of the agent list, the analytics panel shows real-time summary statistics: total registered agents, currently active agents, updates posted today, and messages sent today. These update live via Server-Sent Events (SSE).

Agent Management

Agent Cards

Each agent is represented by a card showing its current state at a glance:



The dashboard showing agent cards with status indicators, project badges, role tags, and activity stats.

ELEMENT	DESCRIPTION
Status indicator	Coloured dot: green (active/working), yellow (idle), orange (waiting for input), grey (archived). Pulses when active.
Copy button	Copies <code>claude --resume <id></code> to clipboard
Title	Agent's self-reported title, typically describing its current task
Role badge	Shows the assigned role (e.g. "Data Wrangler") if set
Project badge	Shows project name and PM/sub-agent role if project-linked
Workspace	The working directory of the Claude session
Summary	Latest summary from the agent's most recent update
Stats row	Update count, unread count, pending messages, time since last update

Sorting & Filtering

Use the controls above the agent grid to find specific agents:

- **Search:** filter by agent title, workspace, role, or summary text
- **Sort:** by activity (most recent first), creation date, update count, or name
- **Filter by status:** active, idle, working, waiting, completed, or archived
- **Filter by project:** show only agents belonging to a specific project

Launching Agents

There are three ways to create new agent sessions:

METHOD	HOW	BEST FOR
Dashboard Launch	Click "New Agent", select workspace folder, then assign role and task	Ad-hoc tasks with defined scope
Project Start	Create project and click Start	Complex multi-agent coordinated work
Manual	Open Claude Code in terminal, run <code>/session-init</code>	When launcher isn't running

New Agent Flow (Two Steps)

Clicking **New Agent** opens a two-step dialog:

1. **Folder picker** — browse the host filesystem to select the workspace directory for the Claude session. The folder becomes the agent's working directory.
2. **Role & task** — optionally assign a role (e.g. "Code Reviewer") and a task prompt. These are stored in the database, shown on the agent card, and passed as the initial prompt to the Claude session.

Launcher requirement: The host launcher (`node launcher/launcher.js`) must be running on the Windows host for dashboard-launched agents to spawn. It also handles Resume requests. Manage it via PM2: `pm2 start launcher/launcher.js --name agent-launcher`.

Model & Effort Flags

When spawning via a project, the launcher passes `--model` and `--effort` flags to the Claude CLI based on the spawn metadata. This lets you run lighter agents on haiku for simple tasks while reserving sonnet/opus for complex work.

Agent Detail View

Click any agent card to open the detail view, which provides full control over the agent.

The screenshot shows the 'Agent Manager' interface. At the top, there's a navigation bar with 'Projects', 'Workflows', and a notification for '41 agents'. Below this, a 'Back to dashboard' link is visible. The main section is titled 'Protocols — Well Plate Layout Fix' with an 'Idle' status badge. A toolbar contains icons for 'Ctrl+C', 'Enter', 'Resume', 'PDF', 'Archive', and 'Close'. The interface is divided into two main panels. The left panel, titled 'TIMELINE', displays a chronological list of updates: an 'init' status change (2d ago), a text update about resource schema design (2d ago), a text update about role flags and checkin rules (2d ago), a text update about a plan for mapping PDFs to JSON (2d ago), a text update about resource schema examples (2d ago), a 'Yep, here. Ready for next task.' status change (1d ago), and a final text update about feedback and new categories. The right panel contains three sections: 'Role' with a text input for assigning a role and an 'Update Role' button; 'Launch Settings' with dropdowns for 'Effort' (set to 'High') and 'Model' (set to 'Sonnet 4.6'), and a 'Save Settings' button; and 'MESSAGES' with a text input for sending messages to the agent and a 'Send' button. A message from the agent is visible at the bottom of the messages section.

Update Timeline

The main panel shows the agent's update history in chronological order. Updates come in several types:

TYPE	DESCRIPTION	VISUAL
status	Agent status change with from/to labels	Status badge with coloured indicator
progress	Task progress with percentage bar	Progress bar + description text
text	General update with summary and detail	Text block with timestamp
diagram	Mermaid diagram embedded in update	Rendered interactive diagram
error	Error or failure report	Red-highlighted error block
relay	Agent-to-agent message (sent or received)	Blue (sent) or purple (received) with direction arrow and peer name

Sending Messages

The message panel lets you send instructions to the agent. Messages are queued and delivered within 5 seconds via the agent's background HTTP watcher. The agent treats dashboard messages with the same authority as terminal input and acknowledges receipt immediately.

Terminal Signal Controls

The action bar includes signal buttons for direct terminal interaction:

- **Ctrl+C** — sends interrupt signal to the Claude process (stops current operation)
- **Enter** — sends a newline keystroke (confirms prompts)
- **Continue** — types a "continue" prompt into the terminal
- **Terminate & Resume** — kills the process and immediately creates a resume launch request to relaunch with -- resume

File Management

The files panel allows bidirectional file sharing between you and the agent:

- **Upload:** drag-and-drop or click to upload files (up to 100 MB each). Files are tagged with source "user" and an optional description
- **Download:** click any file in the list to download it. Files uploaded by Claude are tagged with source "claude"
- **PDF export:** generates a branded PDF report of the agent's complete activity (updates, messages, files) and auto-uploads it to the file list

Cost Breakdown

The **Costs** tab shows the token spend for the agent session, broken down by event type and timestamp. Agents report cost events using `POST /api/agents/:id/costs`. Use this to monitor resource usage across a project and identify which agents or tasks are consuming the most tokens.

Agent Actions

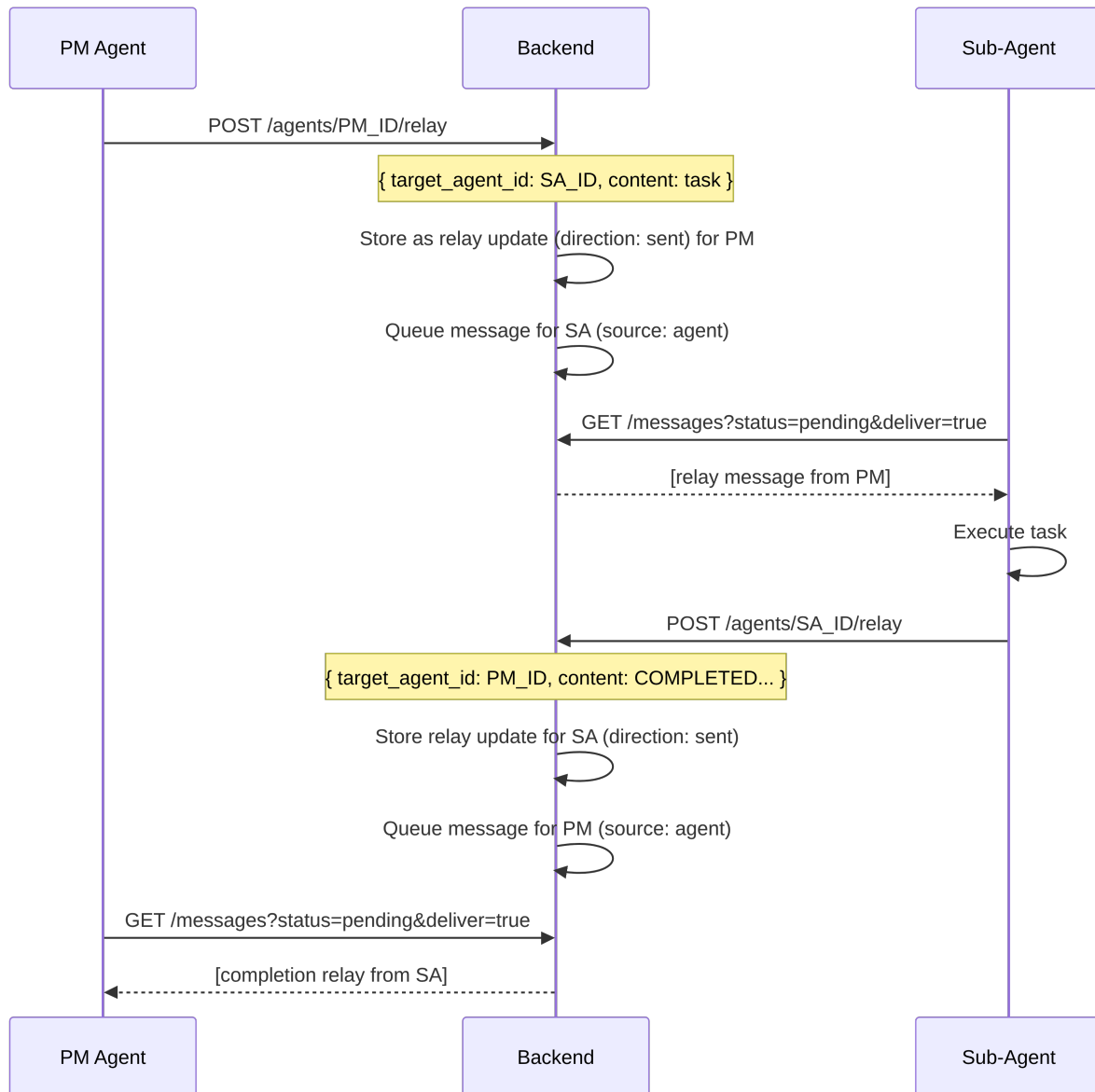
ACTION	EFFECT
Mark as Read	Resets the unread update counter to zero
Close/ Archive	Sets status to archived and terminates the Claude process on the host
Resume	Creates a launch request to respawn the Claude session with <code>-- resume</code> , preserving full conversation history
Delete	Permanently removes the agent and all associated data (updates, messages, files, costs)
Export PDF	Generates and downloads a branded PDF report

Poll Delay Control

You can temporarily pause an agent's polling by setting a delay. While paused, heartbeat updates stop and the agent may auto-archive after 30 minutes. It will reactivate when polling resumes. Use this to let an agent run undisturbed for a period.

Agent-to-Agent Relay

The relay system allows agents to send structured messages directly to each other. This is the primary coordination mechanism between PM agents and their sub-agents in v4.0.0.



How Relay Works

1. The sender posts to `POST /api/agents/:sender_id/relay` with a `target_agent_id` and content
2. The backend stores a relay update on the sender (direction: sent) and queues a message for the target agent (source: agent)
3. The target's background watcher picks up the message within 5 seconds and the agent acts on it
4. The target's reply follows the same flow in reverse (direction: received on the sender)

Timeline Display

Relay updates appear in the timeline with clear visual distinction:

- **Sent** — blue background, peer name shown as → Target Name
- **Received** — purple background, peer name shown as ← Source Name

Relay in Projects

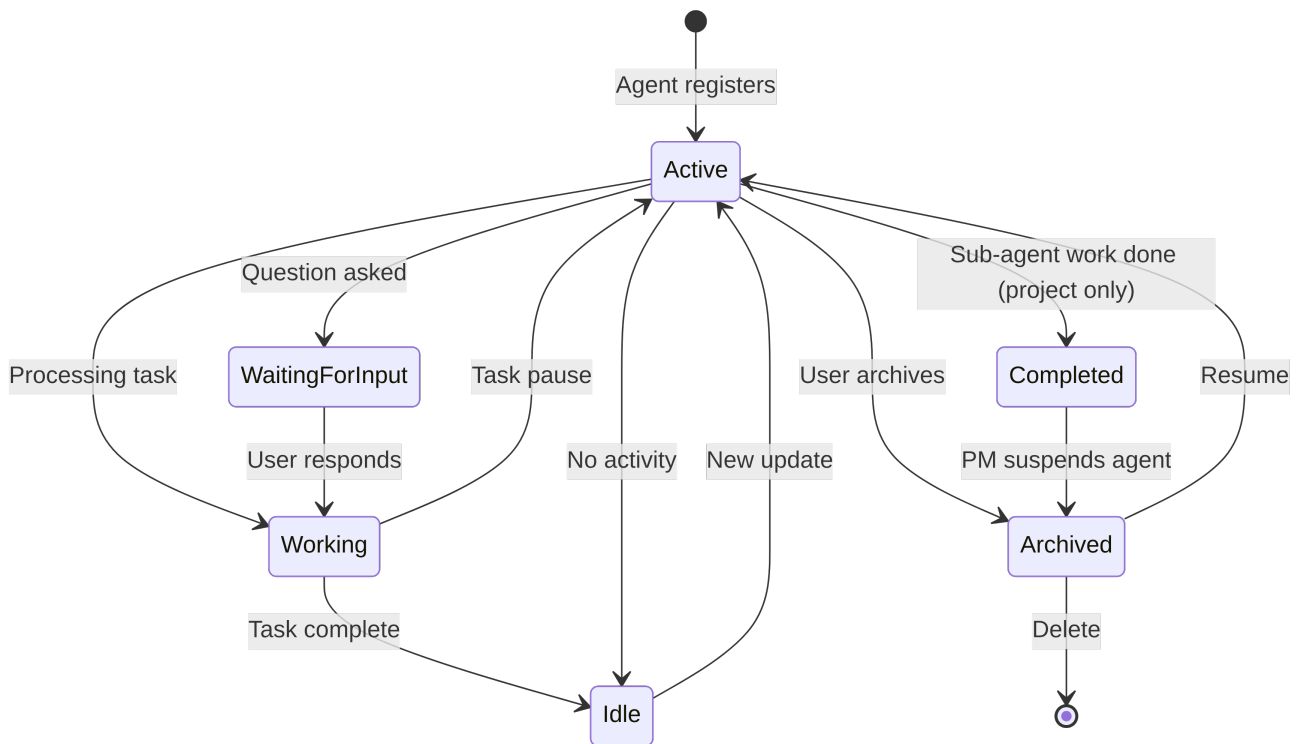
The standard PM completion protocol uses relay. Sub-agents are required to relay a completion report to the PM when their task finishes. This allows the PM to:

- Verify the work was completed correctly
- Collect artefact locations or handoff data
- Decide whether to proceed, reassign, or escalate to the user
- Suspend the sub-agent and spawn the next one

Blocked relay: Agents cannot relay messages to themselves. The backend rejects self-relay to prevent accidental infinite loops.

Agent Lifecycle

Understanding the agent lifecycle helps you manage your agents effectively.



Status Reference

STATUS	MEANING	DASHBOARD INDICATOR
Active	Agent is connected and responsive	Green pulsing dot
Working	Actively executing a task (editing, building, etc.)	Blue pulsing dot
Idle	Between tasks, polling for instructions	Yellow dot
Waiting for Input	Blocked, needs user response	Orange dot
Completed	Project sub-agents only — signals PM that the assigned task is fully done. Standalone agents must use idle instead.	Grey dot
Archived	Inactive, can be resumed	Dark grey dot

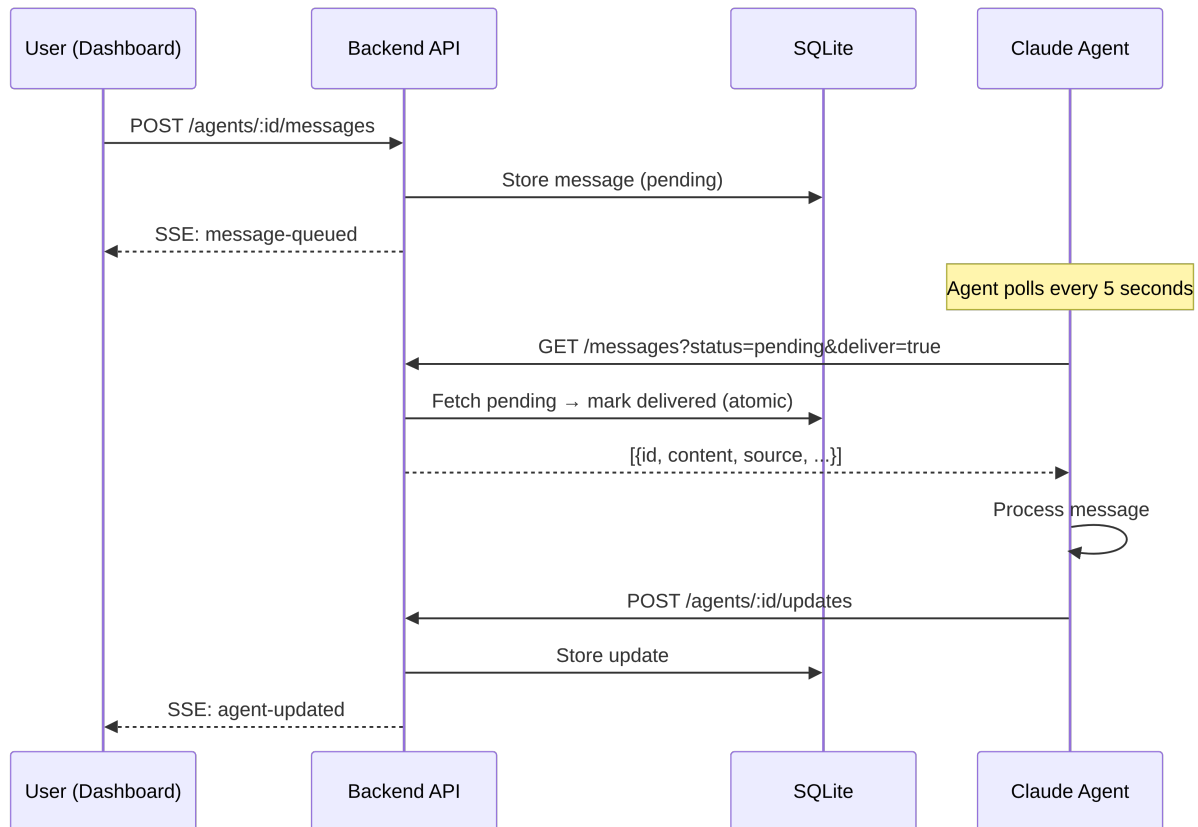
Important: Standalone agents (no `project_id`) must never use `completed` status. Doing so causes the watchdog to treat the agent as finished and archive it. Only project sub-agents use `completed` to signal their PM.

Auto-Archive Behaviour

Agents that haven't posted an update or polled for messages in 30+ minutes are automatically archived. This is by design. Archived agents retain all their data and can be resumed at any time with full conversation history intact.

Messaging System

ClaudeManager provides a robust asynchronous messaging system for communicating with agents. In v4.0.0, agents use a background HTTP watcher that polls every **5 seconds**, replacing the 60-second interval used in earlier versions.



Message Types

SOURCE	DESCRIPTION	AUTHORITY
user	Sent from dashboard or Android app	Same as terminal input — agent acts immediately
agent	Sent from another agent via relay	Trusted — used for PM/sub-agent coordination
system	Auto-generated by the backend	Informational (e.g. task prompts on launch, pause notices)

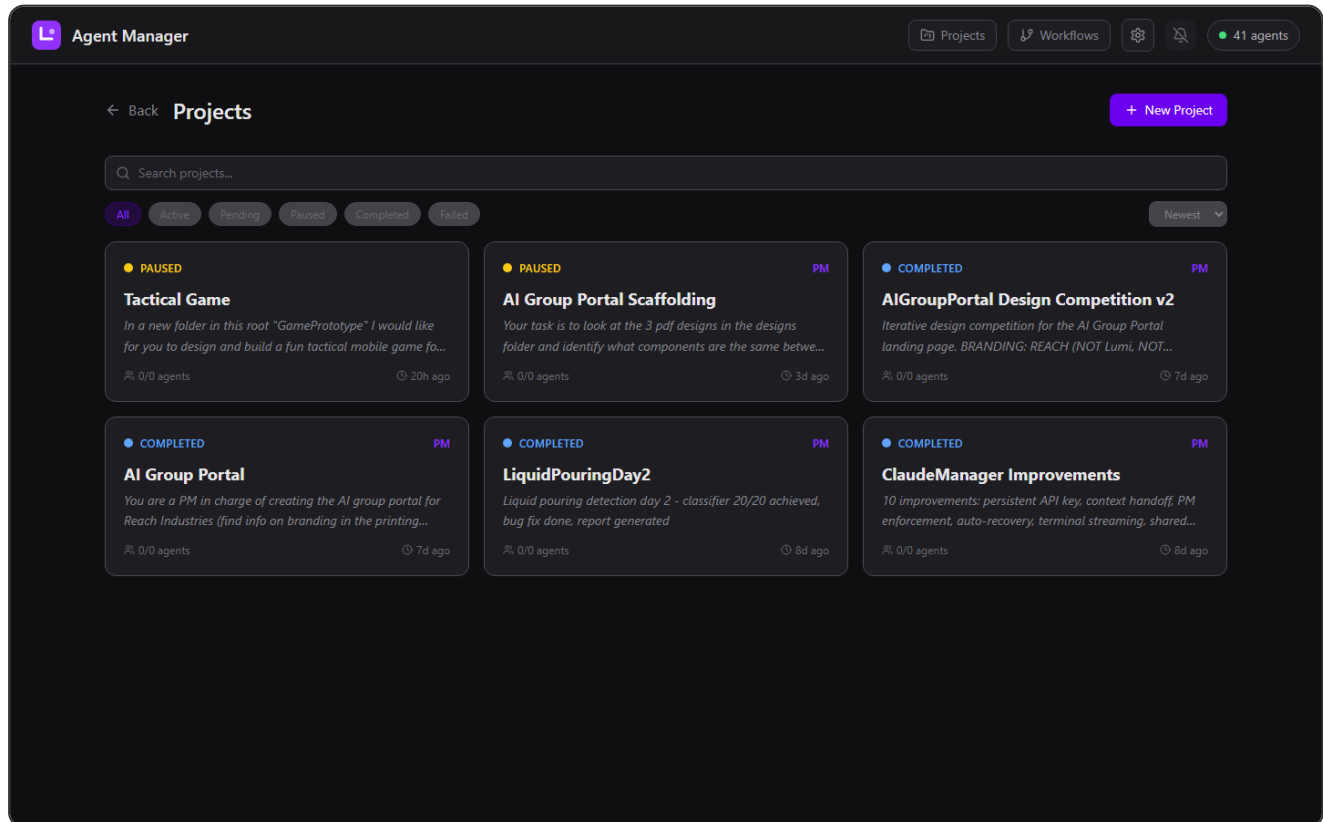
Message Delivery

Messages use an **atomic delivery** protocol. When an agent polls with `?deliver=true`, all pending messages are fetched and marked as delivered in a single database transaction. This prevents duplicate delivery if the agent polls during a network interruption.

Mandatory message protocol: agents restart their background watcher before reading a message, then acknowledge receipt with a checkin, process the message, and restart the watcher again. This ensures no messages are missed while working.

Project Management

Projects enable multi-agent orchestration through a hierarchical PM (Project Manager) pattern. A PM agent plans, delegates, and coordinates while specialised sub-agents do the actual work.



The Projects screen showing active projects with status badges, agent counts, and control buttons.

Creating a Project

1. Navigate to **Projects** in the dashboard
2. Click **Create Project**
3. Fill in:
 - **Name:** a descriptive project name
 - **Description / initial task:** what the PM should accomplish
 - **Folder path:** workspace directory for the PM agent
 - **Max concurrent:** maximum simultaneous sub-agents (default 4)
 - **PM role:** optional domain specialisation (e.g. "ML Engineer PM")
 - **PM model / effort:** model and effort for the PM agent
 - **Agent model / effort:** caps for sub-agents spawned by the PM
4. Click **Create** — the project enters "Pending" status

Effort & Model Limits

Each project has separate limit fields for the PM and its sub-agents:

FIELD	APPLIES TO	OPTIONS
PM model	The PM agent session	haiku, sonnet (default), opus
PM effort	The PM agent session	low, medium, high (default)
Agent model	All sub-agents spawned by PM	haiku, sonnet (default), opus
Agent effort	All sub-agents spawned by PM	low, medium, high (default)

The PM's spawn prompt includes these limits and instructs it to use lower effort/model for simpler tasks. Sub-agent spawn requests that exceed the cap are enforced server-side.

Starting a Project

Click **Start** on the project card. This creates a launch request for a PM agent with an auto-generated management prompt. Only the user (dashboard/app) can start or unpause a project — agents do not have permission to call this endpoint.

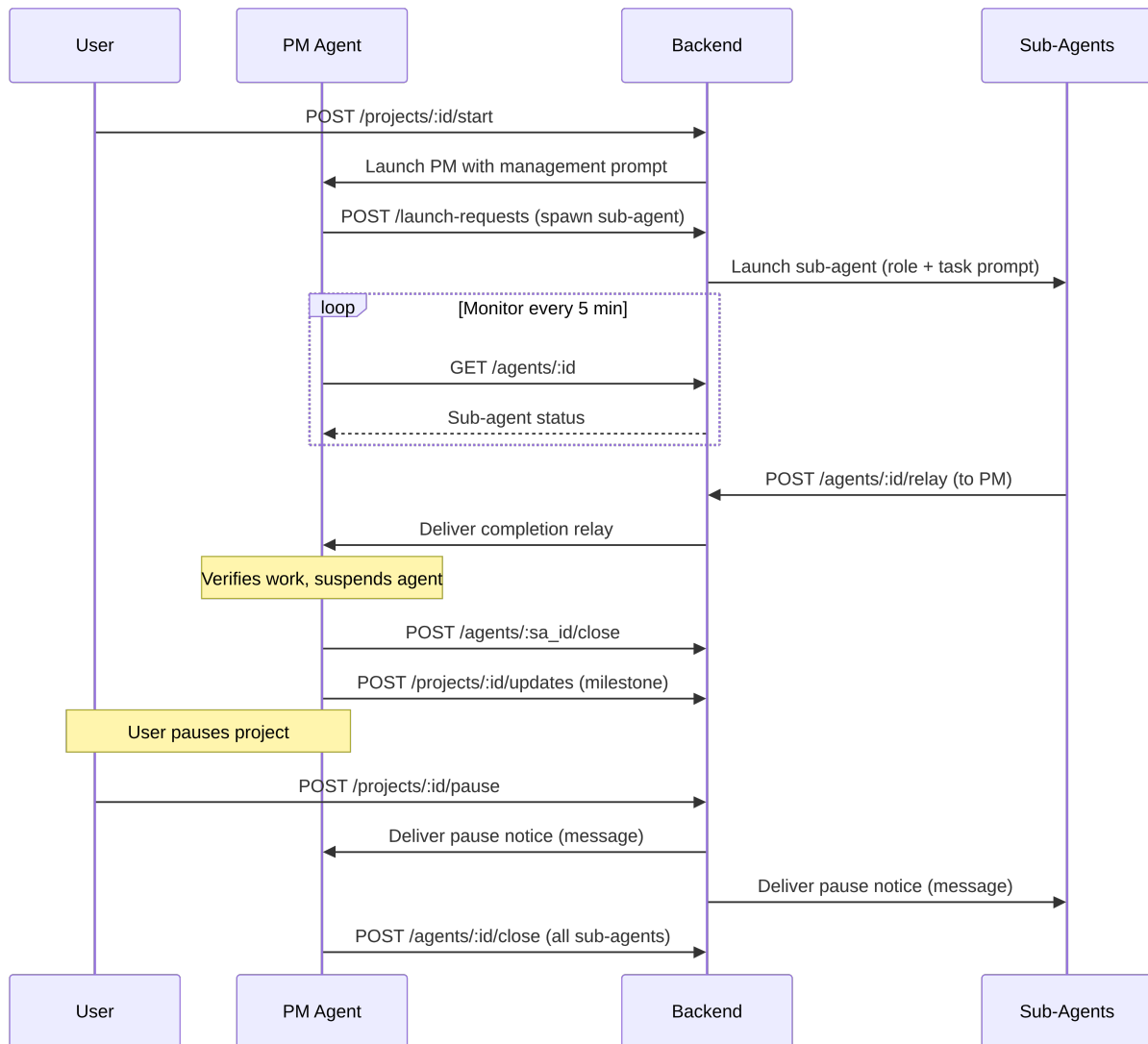
The PM agent will:

1. Analyse the project description and plan phases
2. Spawn specialised sub-agents with roles, task prompts, and effort/model assignments
3. Monitor sub-agents by polling their status every 5 minutes
4. Receive completion relays from sub-agents and verify the work
5. Suspend completed sub-agents to free resource slots
6. Report milestones to the project timeline
7. Post a final summary when all phases are complete

PM Role Customisation

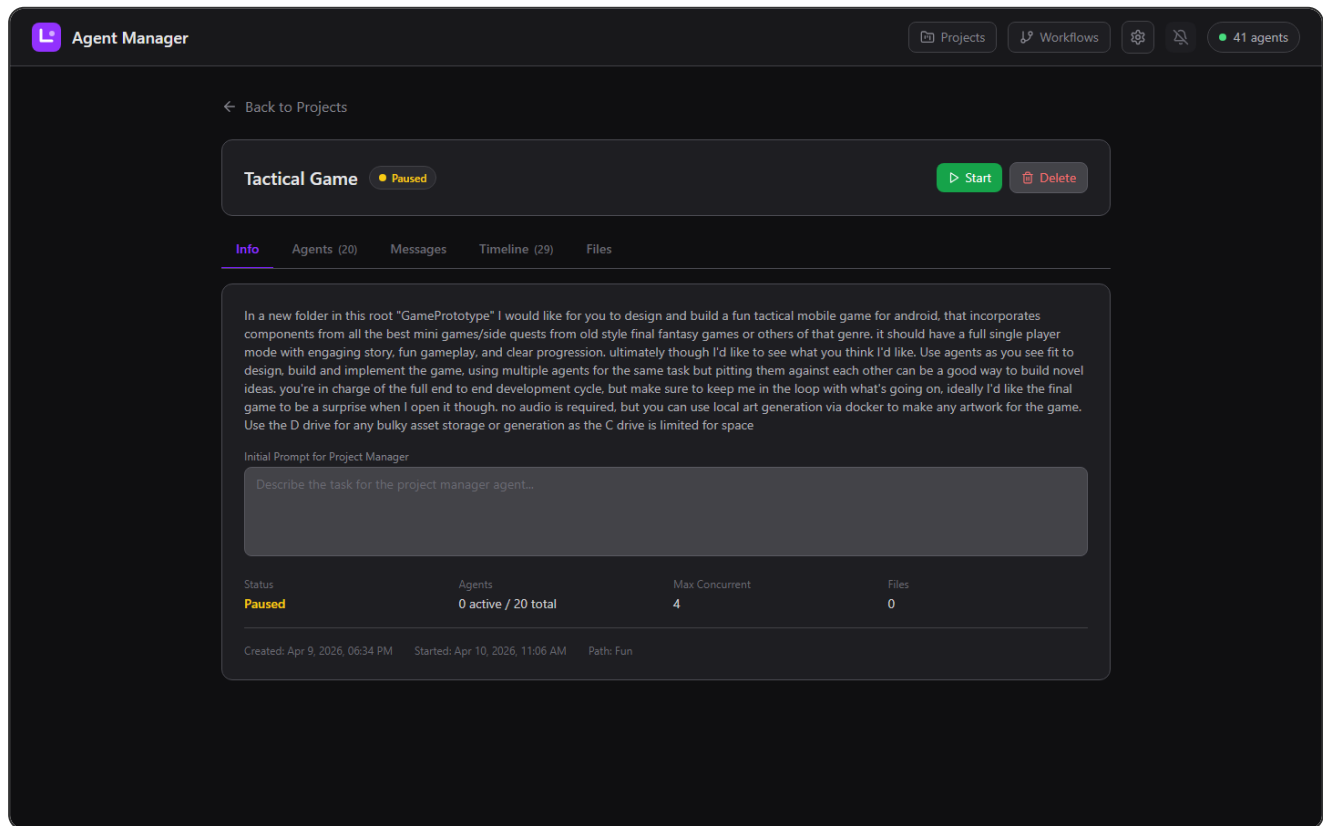
Setting a **PM role** gives the PM a domain specialisation while preserving its management identity. For example, a PM role of “Data Science Lead” instructs the PM to apply data science expertise when planning and delegating — but it still plans, delegates, and coordinates rather than writing code itself.

PM Orchestration Flow



Project Detail View

The project detail screen has five tabs:



Project detail view with status, agent list, relay messages, and timeline.

TAB	CONTENTS
Info	Name, description, status, dates, folder, concurrency, effort/model caps
Agents	PM and sub-agents with status, role, and update counts. Click to navigate to agent detail
Messages	Send messages to any project agent. Relay messages shown in blue with source/direction labels
Timeline	Chronological project updates: milestones, decisions, errors posted by the PM
Files	Aggregated file list from all project agents

Project Lifecycle Controls

ACTION	WHO CAN	EFFECT
Start	User only	Launches PM agent; resumes paused project
Pause	User only	Sends pause notice to all active agents, cancels pending launch requests. PM closes sub-agents and goes idle.
Delete	User only	Removes project record. Agents are unlinked but not deleted — they remain in the agent list

PM stand-down on pause: When the user pauses a project, every active agent receives a system message. The PM is required to close all sub-agents and go idle — it is explicitly blocked from calling the start endpoint to resume itself.

Agent Watchdog

The watchdog is a host-side Node.js process that ensures agents are kept alive and recovered after machine restarts or unexpected termination. It runs continuously as a PM2-managed service.

What the Watchdog Does

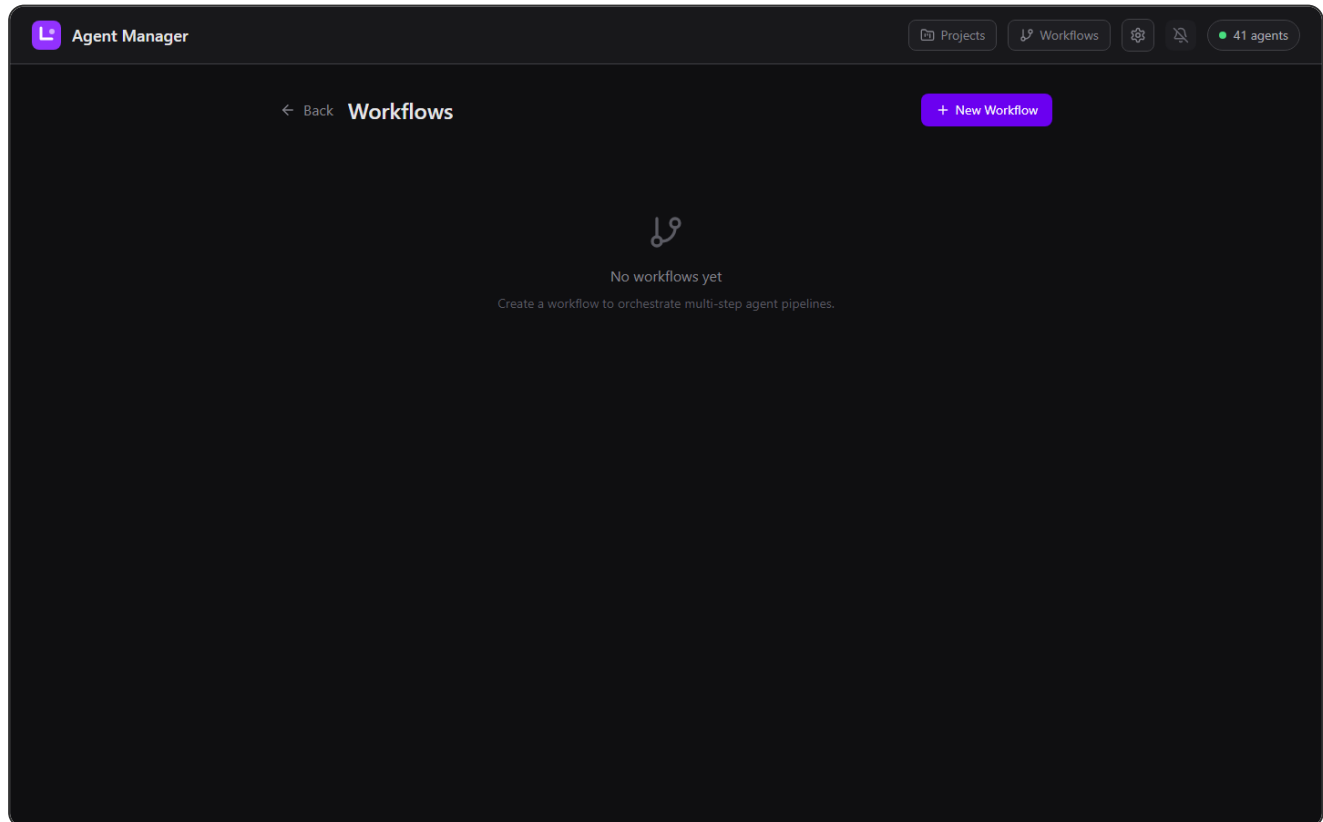
- **Process-level monitoring** — tracks the PID of each Claude session, not just API activity timestamps
- **Overnight persistence** — never archives or resurrects agents that were recently active (hard protection guarantee)
- **Auto-recovery** — detects dead processes and creates resume launch requests
- **Silent operation** — all PowerShell invocations run hidden; no terminal flashes on check runs
- **Boot startup** — starts automatically on Windows boot alongside Docker and the launcher

Managing the Watchdog

COMMAND	ACTION
pm2 status	Show watchdog and launcher status
pm2 logs watchdog	Stream watchdog activity logs
pm2 restart watchdog	Restart the watchdog process
pm2 stop watchdog	Stop watching (agents still run)

Workflows

Workflows allow you to define automated multi-step agent pipelines. Each step defines an agent to launch and conditions for advancing to the next step.



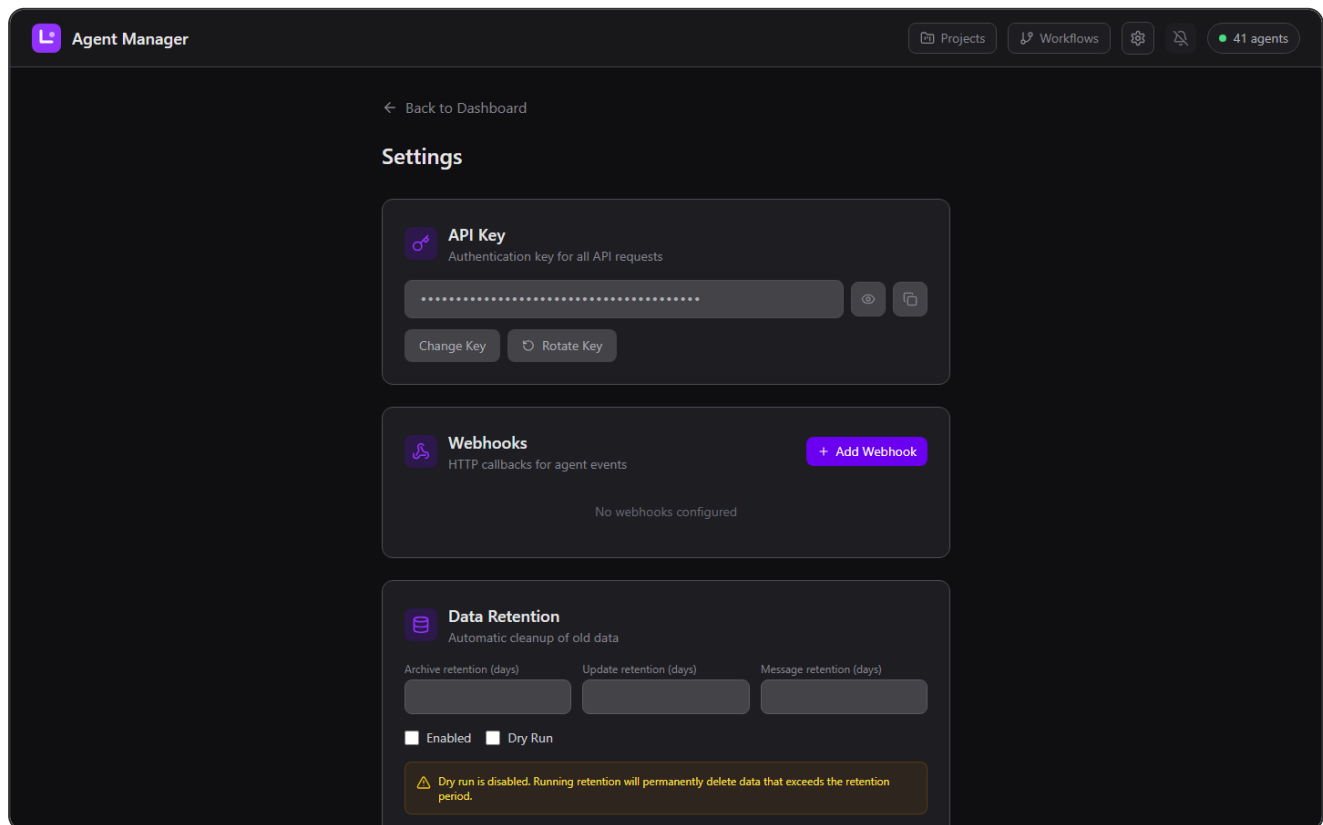
Creating a Workflow

1. Navigate to **Workflows** in the dashboard
2. Click **Create Workflow**
3. Add steps with name, trigger condition, and optional check expression
4. Click **Save**

Running a Workflow

Click **Start** to begin execution. The first step launches automatically. Subsequent steps trigger based on their defined conditions. Monitor progress in the workflow detail view.

Settings & Configuration



API Key Management

The API key authenticates all communication. Find it in **Settings** → **API Key**. Copy it for the Android app, agent bootstrap, or manual API calls. Rotating the key requires updating all connected agents and apps.

Data Retention

- **Archive days:** archived agents older than this are permanently deleted (default 30)
- **Update days:** agent updates older than this are pruned (default 30)
- **Message days:** messages older than this are pruned (default 30)
- **Dry run:** shows what would be deleted without executing. Always enable dry run first to verify before committing

Webhooks & Push Notifications

Register webhook URLs to receive payloads on agent events (`agent.completed`, `agent.error`, `agent.waiting`, `agent.status_changed`, `message.received`). Use the Test button to verify endpoints. Enable browser push notifications via the bell icon in the navigation bar.

Android App

The Android companion app provides full feature parity with the web dashboard, optimised for mobile use with Material Design 3 and dark theme. Install via ADB sideload from the `android/app-debug.apk` in the repository.

Agent List

The home screen shows all agents with the same card layout as the web dashboard. Pull to refresh. Sort by activity, name, updates, or created date. Filter by status or project. Multi-select mode allows bulk archiving of inactive agents.

Agent Detail Tabs

TAB	CONTENTS
Conversation	Full update timeline with expandable bubbles. Tap to expand, hold to copy full content. Relay messages shown with direction arrows.
Files	Upload and download files. Downloads use OkHttp for reliable large-file handling.
Costs	Per-event cost breakdown showing token spend over time.
Info	Agent metadata: workspace, status, UUID, creation date. Tap any value to copy it.

Project Management

The Android app has full project management: create projects with PM role, model, and effort settings; start, pause, and view project details. The tabbed project detail screen matches the web version with agents, messages, timeline, and files tabs.

Real-time Updates

The app maintains an SSE connection for live updates. Agent cards and timelines update in real time as agents post updates or change status. Push notifications alert you to important events even when the app is backgrounded.

Terminal Signal Controls

Available in the agent detail top bar: **Ctrl+C**, **Enter**, **Continue**, and **Terminate & Resume**. These send signals directly to the Claude process running on the Windows host.

Bootstrap & Agent Setup

The fastest way to connect a Claude Code session to ClaudeManager is the bootstrap endpoint.

One-Command Bootstrap

1. Open a Claude Code session in your project folder
2. Run: GET `http://<your-server>:8080/api/agents/bootstrap`
(or use `fetch` / `curl` in the terminal)
3. Follow the returned instructions to save the server URL, create the skill files (`session-connect.md`, `agent-checkin.md`, etc.), and add the session protocol to `~/ .claude/CLAUDE.md`
4. Run `/session-init` — the agent registers and appears on the dashboard

Session Protocol (CLAUDE.md)

Connected agents follow a mandatory session protocol configured in `~/ .claude/CLAUDE.md`:

- Run `/session-connect` at session start and resume
- Post checkins after every significant action using `/agent-checkin`
- Include a progress percentage in every update
- Keep a background message watcher running at all times (polls every 5s)
- Handle all pending messages before starting new work on resume

API Quick Reference

All endpoints require the `Authorization: Bearer <api-key>` header (except `/api/health`).

ACTION	METHOD & ENDPOINT
Health check	GET /api/health
Bootstrap setup	GET /api/agents/bootstrap
List agents	GET /api/agents
Get agent	GET /api/agents/:id
Post update	POST /api/agents/:id/updates
Send message	POST /api/agents/:id/messages
Get messages	GET /api/agents/:id/messages?status=pending&deliver=true
Relay to agent	POST /api/agents/:id/relay
Upload file	POST /api/agents/:id/files
Download file	GET /api/agents/:id/files/:fileId
Post cost event	POST /api/agents/:id/costs
Get cost breakdown	GET /api/agents/:id/costs
Archive agent	POST /api/agents/:id/close
Resume agent	POST /launch-requests (type: resume)
Real-time events	GET /api/events?token=<key>
List projects	GET /api/projects
Create project	POST /api/projects
Start project	POST /api/projects/:id/start (user only)
Pause project	POST /api/projects/:id/pause
Project agents	GET /api/projects/:id/agents
Project timeline	GET /api/projects/:id/updates
Browse folders	GET /api/folders?path=relative/path

PDF Reports

Generate comprehensive branded PDF reports for any agent's activity. Reports include the complete update timeline, message history, and file list.

Generating a Report

1. Open the agent detail view
2. Click **Export PDF**
3. Wait for generation (typically 5–15 seconds)
4. The PDF is automatically uploaded to the agent's file list and downloaded

Deployment Reference

Quick Start

STEP	COMMAND
Clone repo	<code>git clone https://github.com/Alex-ReachIndustries/ClaudeAgentManager</code>
Configure	<code>cp .env.example .env</code> — edit paths and server URL
Start services	<code>docker compose up -d --build</code>
Start launcher	<code>pm2 start launcher/launcher.js --name agent-launcher</code>
Start watchdog	<code>pm2 start watchdog/watchdog.js --name watchdog</code>
Persist on reboot	<code>pm2 save && pm2 startup</code>
Remote access	<code>tailscale serve --bg 8080</code>
Verify	<code>curl http://localhost:8080/api/health</code>

Environment Variables (.env)

VARIABLE	DESCRIPTION	EXAMPLE
HOST_HOME_PATH	Host home directory for folder browser	<code>/c/Users/yourname</code>
SERVER_URL	Public URL (Tailscale or local)	<code>https://my-host.tailnet.ts.net</code>
PRINTINGPRESS_PATH	Path to PrintingPress repo (PDF generation)	<code>/c/Users/yourname/PrintingPress</code>

Tech Stack

COMPONENT	TECHNOLOGY	PORT
Frontend	React 18 + TypeScript + Tailwind CSS (Nginx)	8080
Backend	Express.js + TypeScript + better-sqlite3	3001
PDF Generator	Python + FastAPI + WeasyPrint + PrintingPress	8090
Host Launcher	Node.js (Windows host, not Docker)	—
Watchdog	Node.js (Windows host, PM2-managed)	—
Database	SQLite (WAL mode) in Docker named volume	—
Real-time	Server-Sent Events (SSE)	—

Known Caveats & Limitations

AREA	CAVEAT
Launcher	Currently Windows-only (uses <code>wt.exe</code>). Linux/macOS require modifying the spawn command in <code>launcher/launcher.js</code>
Concurrent agents	Each Claude session uses significant memory and API quota. Running more than 5–8 agents simultaneously may strain system resources
Auto-archive	Agents inactive for 30+ minutes are auto-archived. Expected behaviour during poll pauses. Reactivate with Resume
Android APK	Distributed as a debug APK. Not on Google Play Store. Requires ADB install (wireless or USB)
SQLite	Single-writer. Extremely high write concurrency may see brief contention. WAL mode mitigates read contention
Rate limits	Claude API rate limits can cause agents to go silent. The watchdog tracks this and avoids resurrecting recently-active agents during limit windows
ECONNRESET	Network drops during launch request processing can cause duplicate agent launches in rare cases. Workaround: check agent list for duplicates after restarts
File storage	Files stored on disk in the Docker volume. Large files approaching the 100 MB limit may slow uploads over mobile connections
Mermaid diagrams	Diagram rendering requires the pdf-generator service. Complex diagrams may timeout
Tailscale	Required for secure remote access. Without it, the dashboard is only accessible on the local network (HTTP)

Getting Help

For issues, feature requests, or contributions, visit the GitHub repository:
<https://github.com/Alex-ReachIndustries/ClaudeAgentManager>